

República Bolivariana de Venezuela

Ministerio Del Poder Popular Para La Educación Superior

Universidad de Carabobo

Facultad De Ciencias Y Tecnología



Guía Práctica

Grafos

1. Primera Parte. Dificultad **fácil**

    i. Representación de un grafo (Lista de Adyacencia)
    Dado un número de nodos n y una lista de aristas, implementa una función que construya y muestre la representación del grafo usando una lista de adyacencia.

    ii. Verificación de existencia de arista
    Dado un grafo representado como matriz de adyacencia y dos nodos **u** y **v**, escribe una función que determine si existe una arista directa entre ambos nodos.

    iii. Conteo de grado de un nodo
    Dado un grafo no dirigido y un nodo v, implementa una función que calcule y retorne el grado del nodo (es decir, la cantidad de aristas que lo conectan con otros nodos).

    iv. Recorrido en Amplitud (BFS)
    Dado un grafo no dirigido y un vértice de inicio, implementa el algoritmo de búsqueda en amplitud (BFS). Se requiere mostrar el orden en que se visitan los vértices. Asegúrate de manejar correctamente los vértices ya visitados para evitar ciclos infinitos.

    v. Recorrido en Profundidad (DFS)
    Dado un grafo no dirigido y un vértice de inicio, implementa la búsqueda en profundidad (DFS). Realiza tanto una versión iterativa como una versión recursiva. Muestre el orden de visita.

    vi. Implementación de un Grafo
    Crea una clase o estructura que represente un grafo. Debe permitir dos tipos de implementación: matriz de adyacencia y lista de adyacencia. Incluye métodos para agregar vértices, agregar aristas (tanto dirigidas como no dirigidas) y mostrar la estructura del grafo.

2. Segunda Parte. Dificultad **Media**
    i. Red de amistad con nivel de conexión

    Realiza un grafo no dirigido donde cada nodo representa una persona y cada arista representa una amistad. Implementa las siguientes funciones:

    - agregar_persona (nombre) — agrega un nodo al grafo.
      `func RedAmistad::agregar_persona(nombre: string): void`
    - agregar_amistad (nombre1, nombre2) — conecta dos personas.
      `func RedAmistad::agregar_amistad(nombre1: string, nombre2: string): void`
    - nivel_de_conexion (persona1, persona2) — retorna cuántos intermediarios hay entre dos personas (es decir, la distancia en aristas entre ambos nodos). Si no existe conexión, retorna -1
      `func RedAmistad::nivel_de_conexion(persona1: string, persona2: string): int`

    ii. Sistema de requisitos de asignaturas

    Una universidad necesita verificar que los estudiantes cumplan los prerrequisitos antes de inscribirse en una materia. Modela esto como un grafo dirigido donde una arista de A → B significa que A es prerrequisito de B. Implementa las siguientes funciones:

    - agregar_materia(nombre) — agrega un nodo.
      `func SistemaMaterias::agregar_materia(nombre: string): void`
    - agregar_prerrequisito(materia_previa, materia_siguiente) — agrega una arista dirigida.
      `func SistemaMaterias::agregar_prerrequisito(materia_previa: string, materia_siguiente: string): void`
    - puede_inscribirse(materia, materias_aprobadas) — recibe una lista de materias ya aprobadas y retorna True si el estudiante cumple todos los prerrequisitos directos de la materia objetivo, o False en caso contrario.
      `func SistemaMaterias::puede_inscribirse(materia: string, materias_aprobadas: Lista<string>): bool`

    iii. Mapa de rutas entre ciudades

    Modela un mapa de ciudades como un grafo no dirigido con pesos, donde cada arista representa una carretera y su peso es la distancia en kilómetros. Implementa las siguientes funciones:

    - agregar_ciudad(nombre) — agrega un nodo.
      `func MapaCiudades::agregar_ciudad(nombre: string): void`
    - agregar_ruta(ciudad1, ciudad2, distancia) — agrega una arista con peso.
      `func MapaCiudades::agregar_ruta(ciudad1: string, ciudad2: string, distancia: float): void`
    - ciudades_a_menos_de(ciudad_origen, km_maximos) — retorna una lista de todas las ciudades que se pueden alcanzar desde la ciudad de origen

recorriendo como máximo km_maximos kilómetros en una sola ruta directa.
```
func MapaCiudades::ciudades_a_menos_de(ciudad_origen: string, km_maximos: float): Lista<string>
```

- es_destino_alcanzable(origen, destino) — retorna True si existe al menos un camino (sin importar la distancia total) entre ambas ciudades.
```
func MapaCiudades::es_destino_alcanzable(origen: string, destino: string): bool
```

iv. Camino Más Corto con Peso (Dijkstra)

Implementa el algoritmo de Dijkstra para encontrar la distancia mínima desde un vértice origen a todos los demás en un grafo con pesos no negativos. El programa debe recibir el grafo y el origen, y devolver un arreglo con las distancias mínimas. Optimícelo utilizando cola de prioridad

v. Sistema de recomendación de productos — Medio

Una tienda en línea quiere recomendar productos relacionados. Modela esto como un grafo no dirigido donde los nodos son productos y las aristas indican que dos productos suelen comprarse juntos. Implementa las siguientes funciones:

- agregar_producto(nombre) — agrega un nodo.
```
func SistemaRecomendacion::agregar_producto(nombre: string): void
```
- agregar_relacion(producto1, producto2) — conecta dos productos.
```
func SistemaRecomendacion::agregar_relacion(producto1: string, producto2: string): void
```
- recomendar(producto) — retorna los productos directamente relacionados con el producto dado (vecinos directos).
```
func SistemaRecomendacion::recomendar(producto: string): Lista<string>
```
- recomendar_extendido(producto) — retorna todos los productos relacionados hasta 2 niveles de distancia, sin incluir el producto original.
```
func SistemaRecomendacion::recomendar_extendido(producto: string): Lista<string>
```

vi. Grafo de Rutas con Peajes Variables por Horario

Implementa un grafo no dirigido donde cada arista tiene un costo base, pero también una función que calcula el costo real en función de la hora del día (ej: de 7:00 a 9:00 el costo se duplica, de 17:00 a 19:00 es triple). Se requiere:

- Permitir configurar diferentes funciones de peaje por arista.
- Implementar una función que encuentre la ruta de menor costo desde un origen a un destino considerando que la hora de salida es conocida y el tiempo de viaje por cada arista también es variable (depende de la longitud y la hora).

- La función debe considerar que al llegar a un nodo, la hora actual cambia, afectando los costos de las siguientes aristas.

vii. Árbol genealógico como grafo dirigido

Modela un árbol genealógico usando un grafo dirigido donde una arista de A -> B significa que A es padre/madre de B. Implementa las siguientes funciones:

- agregar_persona(nombre) — agrega un nodo.
  func `ArbolGenealogico::agregar_persona(nombre: string): void`
- agregar_relacion(padre, hijo) — agrega una arista dirigida.
  func `ArbolGenealogico::agregar_relacion(padre: string, hijo: string): void`
- obtener_hijos(persona) — retorna la lista de hijos directos.
  func `ArbolGenealogico::obtener_hijos(persona: string): Lista<string>`
- obtener_ancestros(persona) — retorna todos los ancestros de una persona (padres, abuelos, bisabuelos, etc.) usando un recorrido hacia atrás en el grafo.
  func `ArbolGenealogico::obtener_ancestros(persona: string): Lista<string>`

3. Tercera Parte. Dificultad **Alta**

i. Sistema de detección de fraude Modela una red de transacciones bancarias como un grafo dirigido con pesos, donde los nodos son cuentas bancarias y cada arista dirigida A -> B con peso w representa una transferencia de w dólares de la cuenta "A" a la cuenta "B". Implementa las siguientes funciones:

- agregar_cuenta (id_cuenta) — agrega un nodo.
  ```
  func SistemaFraude::agregar_cuenta(id_cuenta: string): void
  ```
- registrar_transferencia (origen, destino, monto) — agrega una arista dirigida con peso.
  ```
  func SistemaFraude::registrar_transferencia(origen: string, destino: string, monto: float): void
  ```
- detectar_ciclo_de_lavado (cuenta) — detecta si existe algún ciclo que pase por la cuenta dada (indicador de posible lavado de dinero) y retorna los nodos que forman ese ciclo, o una lista vacía si no existe ninguno.
  ```
  func SistemaFraude::detectar_ciclo_de_lavado(cuenta: string): Lista<string>
  ```
- cuenta_mas_activa () — retorna la cuenta con mayor número total de transferencias (entrantes + salientes).
  ```
  func SistemaFraude::cuenta_mas_activa(): string
  ```

ii. Puentes y Puntos de Articulación
Desarrolla un algoritmo que, dado un grafo no dirigido, identifique e imprima todos los puentes (aristas que al eliminarse aumentan el número de componentes conexas) y los puntos de articulación (vértices con la misma propiedad). Para esto, deberás implementar una variante del DFS que utilice conceptos como discovery time y low-link values.

iii. Red eléctrica con detección de fallos
Modela una red eléctrica como un grafo no dirigido donde los nodos son subestaciones y las aristas son líneas de transmisión. Implementa las funciones:

- agregar_subestacion(id) — agrega un nodo.
  ```
  func RedElectrica::agregar_subestacion(id: int): void
  ```
- agregar_linea(sub1, sub2) — agrega una arista.
  ```
  func RedElectrica::agregar_linea(sub1: int, sub2: int): void
  ```
- subestaciones_criticas() — retorna una lista de todas las subestaciones cuya eliminación desconectaría alguna parte de la red.
  ```
  func RedElectrica::subestaciones_criticas(): Lista<int>
  ```
- simular_fallo(subestacion) — elimina temporalmente una subestación y sus líneas del grafo, y retorna cuántos grupos de subestaciones quedan incomunicados entre sí.
  ```
  func RedElectrica::simular_fallo(subestacion: int): int
  ```

iv. Planificador de tareas con dependencias y prioridades

Modela un sistema de gestión de proyectos como un grafo dirigido acíclico (DAG), donde cada nodo es una tarea con un nombre, una duración estimada en horas y una prioridad (1 = alta, 2 = media, 3 = baja). Una arista A → B indica que la tarea A debe completarse antes de que B pueda comenzar. Implementa las siguientes funciones:

- agregar_tarea(nombre, duracion, prioridad) — agrega un nodo con sus atributos.
  ```
  func PlanificadorTareas::agregar_tarea(nombre: string, duracion: int, prioridad: int): void
  ```
- agregar_dependencia(tarea_previa, tarea_siguiente) — agrega una arista dirigida.
  ```
  func PlanificadorTareas::agregar_dependencia(tarea_previa: string, tarea_siguiente: string): void
  ```
- orden_ejecucion() — retorna una lista de tareas en un orden válido de ejecución (orden topológico), respetando todas las dependencias.
  ```
  func PlanificadorTareas::orden_ejecucion(): Lista<string>
  ```
- tiempo_minimo_proyecto() — calcula el tiempo mínimo total para completar todas las tareas, considerando que tareas sin dependencias entre sí pueden ejecutarse en paralelo (ruta crítica del DAG).
  ```
  func PlanificadorTareas::tiempo_minimo_proyecto(): int
  ```
- tareas_bloqueadas(tarea) — retorna todas las tareas que no pueden iniciarse hasta que la tarea indicada sea completada (directa o indirectamente).
  ```
  func PlanificadorTareas::tareas_bloqueadas(tarea: string): Lista<string>
  ```

v. Grafo de Optimización de Recursos con Restricciones Geográficas

Realiza un grafo donde los nodos representan ubicaciones geográficas con una demanda de recursos (positiva si produce, negativa si consume) y las aristas tienen costo de transporte por unidad. Implementa:

- Un algoritmo que determine cómo redistribuir recursos entre nodos para satisfacer todas las demandas minimizando el costo total de transporte.
- El algoritmo debe considerar que los nodos pueden tener límites de almacenamiento.
- Si no es posible satisfacer todas las demandas, el algoritmo debe identificar qué demandas no pueden cumplirse y sugerir qué nodos deberían aumentar su producción.
- Este problema es una variante del problema de flujo de costo mínimo con demandas y capacidades de nodo.